Report and code: Prompt design and the energy cost of correct code

A consumer-CPU experiment with two small Qwen2.5-Coder models

Research report | Revised 28 October 2026

Why this experiment matters

An AI-generated answer has a physical cost: computation, time and energy. For local users, efficiency affects whether useful AI can run within the resources available to them. Across repeated requests, differences in resource use can accumulate. Efficiency therefore deserves attention alongside accuracy and usability [1,2]. Prompt design is one accessible control because instructions can change response length and the likelihood of obtaining usable output. The relevant question is not simply how to generate fewer tokens, but how to obtain a correct answer at a reasonable resource cost.

This report takes a small first step: 40 programming tasks, seven prompt variants, two small models and one consumer laptop. It measures tokens, latency, test success and estimated CPU package energy, with substantial baseline-drift limitations. It does not establish battery-life improvements, monetary or carbon savings, or a universally efficient prompt. Its value is to make resource costs visible, examine them alongside verified success, and identify better-controlled questions for future research.

Abstract

We ran this experiment to find out how much a prompt can change the cost of getting correct code from a small local model. We used seven prompt variants on the same 40 HumanEval tasks with Qwen2.5-Coder 0.5B and 1.5B, giving 560 main responses and 280 additional responses for repeat checks. Asking for explanations produced longer answers and higher estimated energy use, but did not improve the overall pass count compared with the short code-only instruction. Examples gave mixed results: three examples reduced output length compared with one example, yet passed two fewer tasks on each model. We also found that the cheapest response was not always the cheapest route to a correct answer. These are observations from one laptop and two small models. All main-run energy estimates were affected by idle-baseline drift, and accuracy differences remain uncertain. The report therefore provides a starting point for further experiments rather than a general rule about the best prompt.

1. Motivation and related work

For a local user, an answer consumes time, memory and computation whether or not it passes a test. Comparing energy per response alone can reward a prompt that produces inexpensive but incorrect code. We therefore examine both per-attempt cost and aggregate energy per correct solution, including energy spent on unsuccessful attempts. This ratio is a measure of verified yield across a task set, not a measured retry policy or a claim about financial savings.

Green AI argues that efficiency should accompany accuracy as an evaluation criterion [1]. Poddar et al. report associations between inference energy, output length and response time,

and investigate prompting and system settings [2]. The present experiment is a small application of these questions to testable code generation on a consumer CPU. It does not claim a novel energy meter, a new model, or the first demonstration that response length matters. HumanEval supplies programming tasks and executable checks [3]; Qwen2.5-Coder supplies the two code-oriented models [4].

The report compares task-completion instructions, example-guided prompts, and prompts that request explanations before code. We examine output length, test success and resource cost together. Some comparisons change more than one feature: P3 adds examples and changes the instruction wording. P1 versus P0 is a narrower comparison because it adds only “Return only code.” Keeping these differences visible helps us avoid attributing every result to a single prompt feature.

2. Experimental setup and protocol

The host was a Windows 11 laptop (build 26300) with an Intel Core i5-13420H, 16 GB RAM and CPU-only inference. Python was 3.13.7 (64-bit); Ollama was 0.40.6. Both qwen2.5-coder:0.5b and qwen2.5-coder:1.5b used Q4_K_M quantization. Registered settings were temperature 0, seed 1729, eight CPU threads, num_ctx=4096, num_predict=768 and num_gpu=0. There was one client request at a time, no custom stop strings and no conversational history. A 4096-token context is a capacity setting, not a fixed-length input; stage-1 API input counts ranged from 74 to 471. The 768-token output setting is a maximum, not a target length.

The common system text was “You are a helpful coding assistant.” The user prompt joined the instruction, any original demonstrations, and the unchanged task signature/docstring. Tests and reference solutions were not provided. The table gives exact instruction text except the separately quoted P5.

Arm	Instruction	Added examples
P0	Complete the Python function.	0
P1	Complete the Python function. Return only code.	0
P2	Explain your solution, then provide the completed Python function.	0
P3_1	Complete the Python function following the example.	1
P3_3	Complete the Python function following the examples.	3
P4	Reason step by step, then provide the completed Python function.	0
P5	Long code-only instruction, quoted below	0

P5: “Please read the following Python function signature and docstring carefully. Complete the function so that it satisfies the docstring and handles edge cases correctly. Return only the code for the completed function, with no explanation.” Demonstrations implement a dictionary lookup, label wrapping and tab counting; their exact code and whitespace are retained in docs/PROMPTS.md. They were checked against the selected tasks before generation.

Forty tasks were sampled without replacement with seed 1729 from 164 HumanEval tasks. Stage 1 comprised 40 tasks x seven arms x two models x one repeat. Stage 2 used a previously frozen ten-task subset with two extra repeats (280 requests), for noise diagnostics only. Within-stage order was seeded and interleaved. Stage 1 is not pooled with stage 2, which would overweight ten tasks. Separate calibration froze 26 solved and 14 failed labels; reference-length grouping froze 21 short and 19 long tasks at a six-line median. P0 is excluded from group-wise comparisons.

Native Ollama API fields supplied input/output token counts and durations [5]. A saved warm-up was excluded; scored requests used keep_alive=0 under the conservative cache policy, unloading between calls and recording load duration. Earlier probe text was not identical at fixed settings; we do not promise deterministic generation. HWiNFO 8.54-6090 logged CPU Package Power every 500 ms [6]. AC power and Best performance were checked; the intended protocol also required sleep disabled and unnecessary apps closed. The experiment did not independently rule out every background process.

3. Measurement, scoring and analysis

Gross post-load energy is the trapezoidal integral of CPU Package Power between request start plus Ollama load_duration and response end, expressed in Wh (1 Wh = 3600 J). This boundary is approximate: an aggregate API duration is not a synchronized phase timestamp. Registered net energy subtracts the block-start idle power multiplied by the same measured window duration. Both gross and net are reported. CPU package energy is not whole-laptop electricity, RAM energy, an API bill, or carbon emissions. All pilot prompt-evaluation windows were low-resolution; no input/output phase-energy attribution is made.

Blocks stopped launching requests after three hours with a four-hour ceiling, and used five-second request gaps and 60-second idle measurements at both ends. All stage-1 requests occurred in the first block: starting idle was 17.553 W and ending idle 11.516 W. The difference exceeds the registered drift threshold max(1 W, 10% of start); all 560 estimates therefore retain a drift flag. CodeCarbon was removed from scored full runs because its pilot estimates were constant and did not corroborate workload-dependent power.

The extractor chose the last complete python/py/unlabelled fenced block that parsed and defined the target function, otherwise the stripped response. Trusted task prelude was prepended. Generated code ran only in the existing restricted Windows subprocess with a five-second deadline, 256 MiB limit, one-process Job Object, minimal environment, temporary directory and AST/import checks. This is not an OS security boundary. All 40 references passed this evaluation path. The fixed cap rule marks eval_count >=768 or done_reason=length as truncated before testing, even if a complete function appears earlier. Passing means passing these benchmark tests, not proof of general correctness.

The original H1-H5 concern output-length association, concise-output cost/accuracy, examples, step-by-step prompts, and longer code-only instructions. Original per-arm summaries and low-resolution sensitivity remain registered analyses. The five paired contrasts, ratio intervals, failure review, failed-attempt energy share and alternate-baseline scenarios below were requested after results were seen. They are explicitly post hoc. We do not retroactively call them pre-specified hypothesis tests.

Within each model, we align the same 40 tasks across arms. For each contrast A-B, we bootstrap task pairs 2,000 times with seed 1729, calculating mean net-energy differences and mean binary-outcome differences. We report 2.5th/97.5th percentiles and discordant pass counts. These are pointwise intervals, with no multiplicity adjustment. Per-arm cost-per-correct intervals resample tasks and recompute sum(energy)/sum(passed), including failures in the numerator. No zero-success resample occurred in these data; the script marks intervals unavailable if any occur rather than silently dropping draws. Task resampling does not account for systematic drift or missing repeat evidence.

For the baseline sensitivity, the same saved gross integrals and post-load seconds are evaluated against constant end and midpoint baselines. These scenarios change an assumption; they neither reconstruct the idle trajectory nor replace the registered estimates. The pilot contributes supplementary repeat diagnostics only.

4. Results: paired comparisons across prompt variants

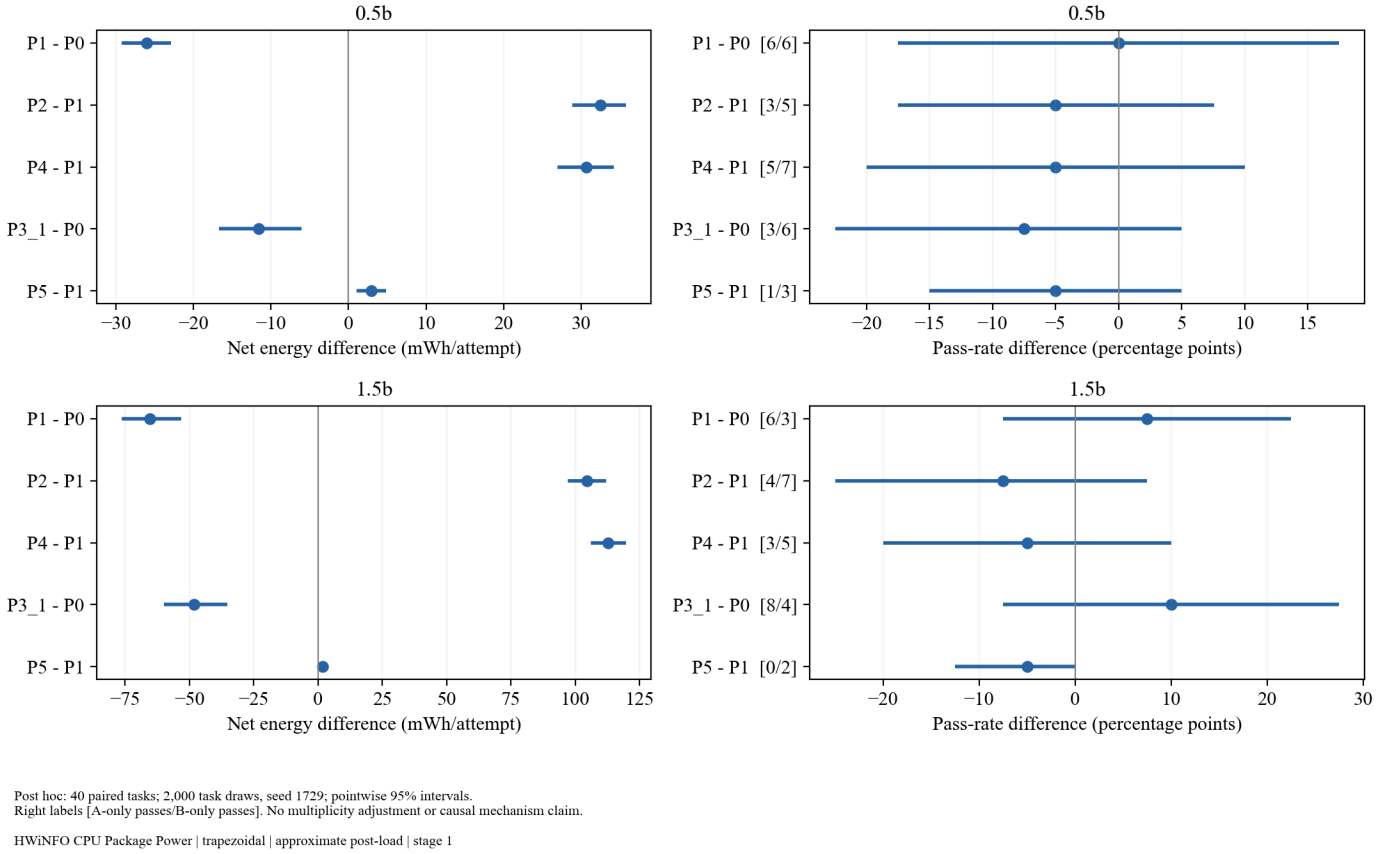


Figure 1. Task-paired differences (A minus B) in stage-1 net energy and pass rate. Lower energy differences favor A; positive pass-rate differences favor A. Bracketed counts show A-only/B-only passes. Post-hoc, pointwise 95% bootstrap intervals; n=40 task pairs per contrast. HWiNFO CPU package estimates, approximate post-load, registered start-baseline adjustment.

Model	A minus B	Net difference [95% interval], mWh	Pass difference [95% interval], pp	A-only / B-only
0.5b	P1 - P0	-26.04 [-29.23, -22.93]	0.0 [-17.5, 17.5]	6 / 6
0.5b	P2 - P1	32.52 [28.85, 35.77]	-5.0 [-17.5, 7.6]	3 / 5

Model	A minus B	Net difference [95% interval], mWh	Pass difference [95% interval], pp	A-only / B-only
0.5b	P4 - P1	30.67 [26.97, 34.25]	-5.0 [-20.0, 10.0]	5 / 7
0.5b	P3_1 - P0	-11.60 [-16.73, -6.05]	-7.5 [-22.5, 5.0]	3 / 6
0.5b	P5 - P1	2.91 [1.03, 4.87]	-5.0 [-15.0, 5.0]	1 / 3
1.5b	P1 - P0	-65.47 [-76.27, -53.15]	7.5 [-7.5, 22.5]	6 / 3
1.5b	P2 - P1	104.70 [97.22, 111.97]	-7.5 [-25.0, 7.5]	4 / 7
1.5b	P4 - P1	112.76 [106.06, 119.76]	-5.0 [-20.0, 10.0]	3 / 5
1.5b	P3_1 - P0	-48.12 [-59.96, -35.27]	10.0 [-7.5, 27.5]	8 / 4
1.5b	P5 - P1	1.69 [-0.30, 3.95]	-5.0 [-12.5, 0.0]	0 / 2

P1 reduced net energy versus P0 by 49.5% for 0.5B and 73.5% for 1.5B. Equal 0.5B pass totals conceal six tasks gained and six lost; the outputs are not interchangeable. For 1.5B, six gained and three lost produce a three-task increase. The intervals permit accuracy changes in either direction. P2 and P4 have substantially higher energy than P1 in these samples, while neither has a higher aggregate pass count. P3_1-P0 points toward lower energy for both models but opposite accuracy directions. Because P3 also changes wording, this is not a causal estimate of examples alone.

5. Cost per correct solution and per-attempt outcomes

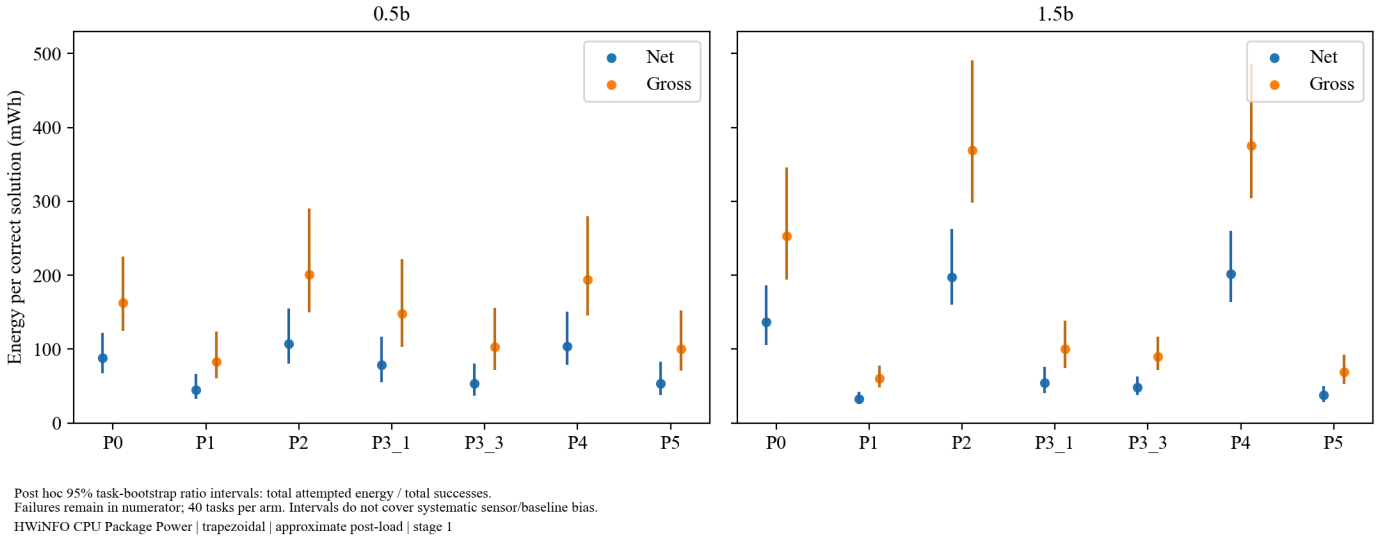


Figure 2. Aggregate gross/net energy per correct solution with task-bootstrap ratio intervals, n=40 attempts per arm. The numerator includes failed attempts. HWiNFO CPU Package Power, trapezoidal integration, approximate post-load; net uses the starting baseline. Overlap of these per-arm intervals is not a paired significance test.

Model	Arm	Pass / 40	Output tokens	Net mWh	Gross mWh
0.5b	P0	24	462.2	52.65	97.91
0.5b	P1	24	227.5	26.61	49.74

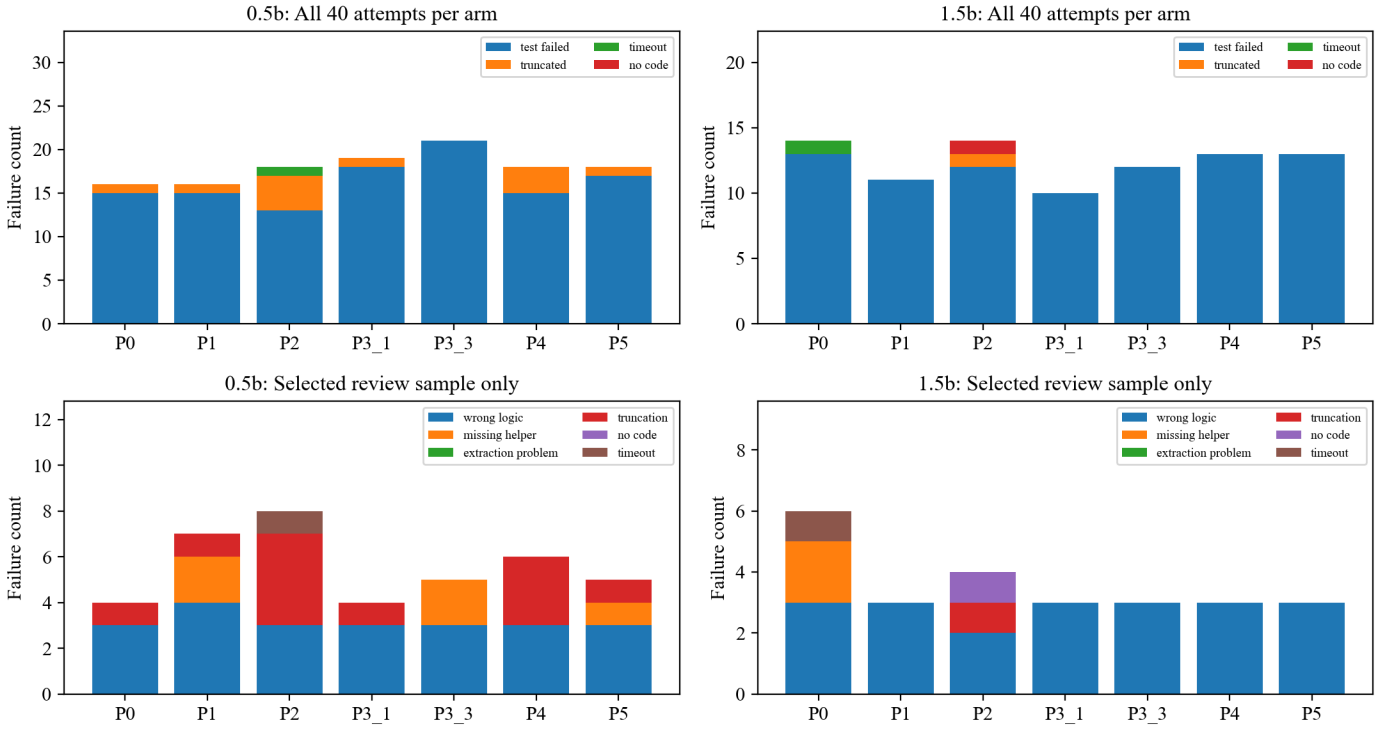
Model	Arm	Pass / 40	Output tokens	Net mWh	Gross mWh
0.5b	P2	22	531.0	59.13	110.66
0.5b	P3_1	21	357.8	41.05	77.54
0.5b	P3_3	19	194.2	25.18	48.68
0.5b	P4	22	520.4	57.28	106.80
0.5b	P5	22	243.6	29.52	55.06
1.5b	P0	26	336.1	89.06	164.73
1.5b	P1	29	71.1	23.59	43.59
1.5b	P2	26	500.1	128.29	239.78
1.5b	P3_1	30	130.2	40.94	75.06
1.5b	P3_3	28	98.2	33.84	63.02
1.5b	P4	27	526.5	136.34	253.52
1.5b	P5	27	73.6	25.27	46.68

Table 1. Stage-1 means over all attempts. Energy is estimated CPU package energy (HWiNFO), not whole-device energy. Every energy row carries the baseline-drift caveat. P1 has the lowest observed net and gross cost per correct solution for each model: 44.34 and 82.91 mWh for 0.5B; 32.53 and 60.13 mWh for 1.5B. The net 95% intervals are [32.23, 66.30] and [25.74, 41.81] mWh. These are sample rankings, not proof of a universally best prompt.

For 0.5B, P3_3 uses the fewest output tokens and lowest mean energy per attempt, but passes only 19 tasks. For 1.5B, P3_1 has the highest observed pass count (30), one above P1; that single-task difference is uncertain. A request-level saving and a cost-per-correct saving are different objectives.

6. Failure mechanisms and the output cap

Stage 1 recorded 347 passes and 213 failures: 198 test_failed, 12 truncated, two sandbox timeouts and one no_code. P2 hit the 768-token cap in 4/40 (10%) 0.5B requests and 1/40 (2.5%) 1.5B requests. P4 hit it in 3/40 (7.5%) 0.5B requests and 0/40 1.5B requests. Cap failures therefore cannot explain all explanation-prompt failures; all 13 failures for 1.5B/P4 were test failures.



Top: machine outcomes for every stage-1 failure. Bottom: 64 selected failures, AI-assisted reading.
Zero observed extraction problems in sample is not a complete extraction audit; no outcomes changed.

HWiNFO CPU Package Power | trapezoidal | approximate post-load | stage 1

Figure 3. Top panels count recorded machine failure types across all 40 stage-1 requests per arm. Bottom panels show mechanisms in the selected 64-case review sample, not estimated population prevalence. No scoring rules or outcomes were changed.

The AI-assisted review selected the first three failures by run ID within each model/arm, plus every stage-1 cap/no-code case and every saved NameError/timeout case. The resulting 64 cases comprise 42 wrong-logic cases, seven missing-helper/import cases, 12 cap failures, two timeouts and one prose-only response. No extraction problem was identified in this sample. This is not an exhaustive independent human audit; sample selection enriches particular failures and prevents estimating prevalence from these proportions. Annotations were checked against saved source text; an independent second review was not performed.

Concrete examples include full-0004 (splits only comma-space, leaving ordinary words unsplit), full-0054 (ignores operator precedence), full-0171 (calls math.ceil without importing math), and full-0460 (undefined is_prime helper). In full-0452 the raw response contains prose and no target function: the extractor did not discard a valid implementation. Some capped responses contain parseable code followed by unfinished explanation; cap failure is a protocol outcome, not necessarily incomplete code. Tests were not rerun to revise those outcomes. Per-run evidence, reasons and text hashes are in P4_FAILURE_REVIEW.csv; raw and extracted text can be inspected with experiments.show_run.

7. Failed-attempt energy and selection effects

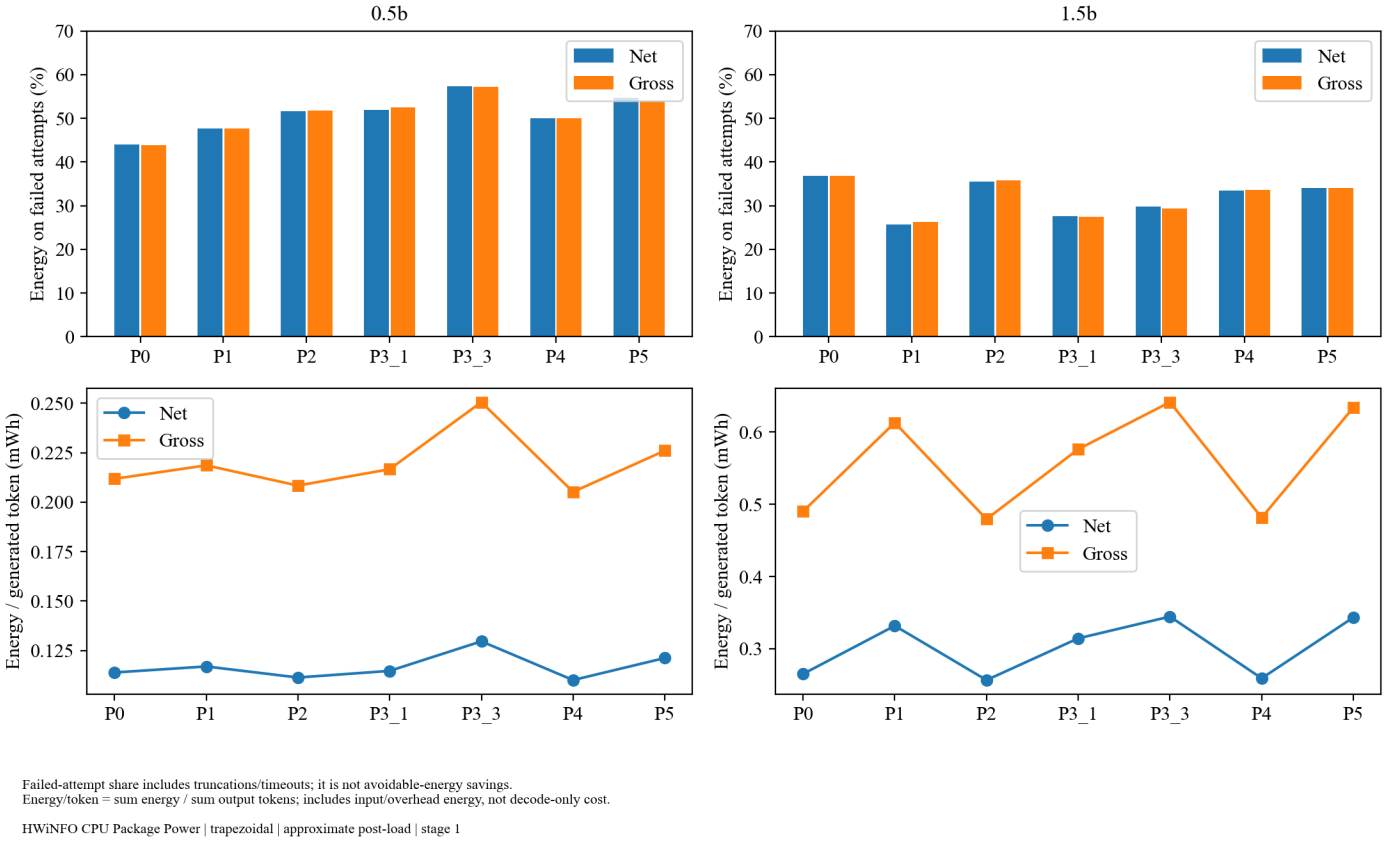


Figure 4. Top: fraction of total attempted energy spent on nonpassing responses, gross and registered net. Bottom: ratio of total energy to total generated tokens. Both retain all stage-1 attempts and use HWiNFO CPU package estimates. Energy per output token includes prompt-processing and overhead energy; it is not a direct decoding-energy measurement.

The “wasted energy” label refers only to energy spent on failed attempts, not energy known to be avoidable. Net shares span 44.0-57.3% across 0.5B arms and 25.8-36.9% across 1.5B arms. P1's shares are 47.7% and 25.8%, respectively. A prompt can reduce absolute cost while having a larger failed-energy fraction: 0.5B/P1 uses much less total energy than P0 but has a slightly larger failed-energy share. It is important to report the denominator and absolute cost together.

The registered low-resolution sensitivity removes 70 of 560 requests because their post-load windows are shorter than three seconds. Removal is not random: it preferentially excludes the shortest, cheapest outputs. For 1.5B/P1, only 24/40 remain; P2 and P4 retain all 40 each. Thus this view changes task composition and does not provide an unbiased correction. The retained P1/P5 mean-net ordering reverses on 1.5B. We preserve the view as a limitation check, not evidence that exclusion improves measurement truth.

Energy per generated token offers a different perspective but cannot repair that selection bias. For 1.5B, P1 costs 0.332 mWh net per generated token versus 0.259 for P4, despite being much cheaper per attempt. Short outputs amortize prompt-processing and other overhead over fewer tokens. This is a plausible explanation, not a measured phase decomposition. Optimizing energy per token alone could encourage unnecessary output and conflict with the user's objective of a correct answer.

8. Baseline-drift sensitivity and repeat evidence

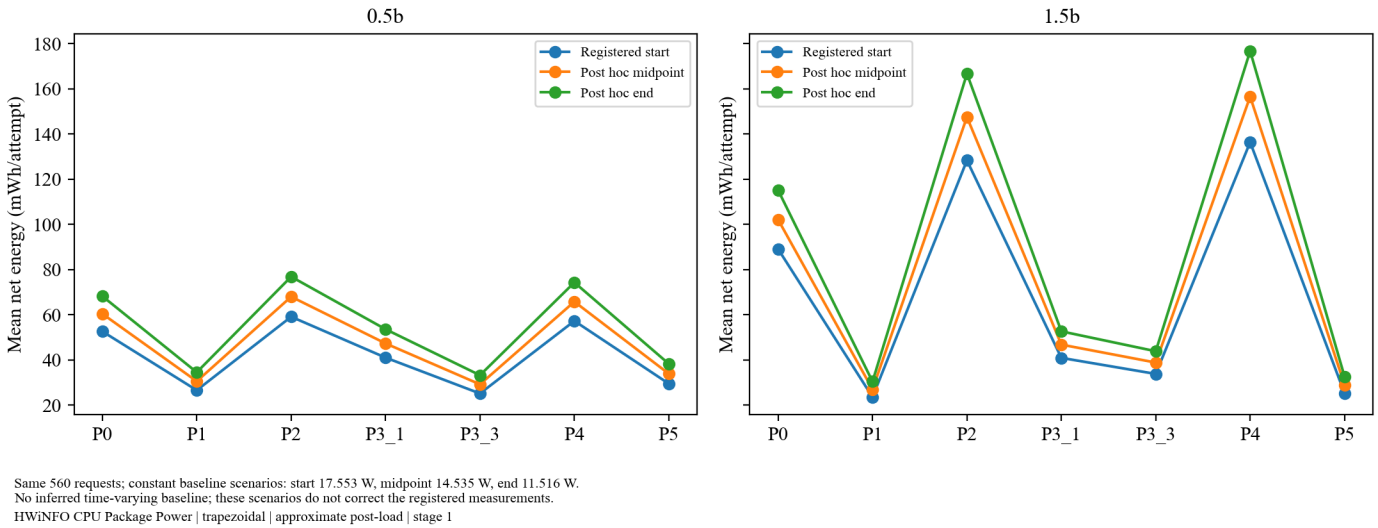


Figure 5. Registered start-baseline net beside post-hoc midpoint/end-baseline net. Constant baselines are 17.553, 14.535 and 11.516 W, respectively, applied to the same saved windows and gross integrals. No scenario is asserted to be the true baseline during generation.

Using the lower end baseline raises estimated net energy. P1's mean changes from 26.61 to 34.56 mWh for 0.5B and from 23.59 to 30.47 for 1.5B. P0 changes from 52.65 to 68.21 and 89.06 to 115.08 mWh. The P1-P0 direction remains the same under these constant-baseline scenarios, but neither the bootstrap intervals nor these scenarios remove drift-related uncertainty. The midpoint is a sensitivity assumption, not evidence that drift was linear.

Stage 2 contains only 64/140 complete energy pairs, with 181/280 energy values. Missingness followed interrupted blocks; reliable arm-specific variance estimates cannot be claimed. The separate 320-request pilot provides 159/160 complete energy pairs, summarized below without pooling into stage 1.

Model	Arm	Pairs	Median absolute difference (mWh)	90th percentile (mWh)
0.5b	P0	20/20	2.46	10.51
0.5b	P1	20/20	2.84	5.65
0.5b	P2	20/20	3.40	10.40
0.5b	P4	20/20	3.45	9.22
1.5b	P0	20/20	4.44	8.62
1.5b	P1	20/20	1.42	4.66
1.5b	P2	20/20	4.34	9.29
1.5b	P4	19/20	14.75	20.67

Table 2. Supplementary pilot only: absolute differences between two net post-load HWiNFO estimates on the same task/model/arm. The pilot used a 15.420 W baseline, only

four prompt arms, and active CodeCarbon measurement overhead; its conditions differ from the full run. The pilot is excluded from hypothesis tests. All complete pairs in these pilot and stage-2 summaries have equal output-token counts, but this alone does not prove identical text or identical system state. Spreads reflect operating conditions and measurement variation; they are not precise isolated sensor-noise estimates. Small P1-P5 differences are comparable to some observed repeat spreads, motivating caution.

9. Expectations, interpretation and limits

H1 predicted an association between output length and energy/latency. The results are consistent with it: P2/P4 produce far longer responses and higher request energy. Existing joint input/output regressions have in-sample net R-squared values of 0.974 (0.5B) and 0.993 (1.5B). Joint fit does not identify each predictor's contribution or establish causality. The experiment cannot directly allocate energy to input processing versus decoding.

H2 anticipated lower cost from concise outputs, with possible accuracy differences across task groups. P1-P0 supports the cost observation; the paired accuracy intervals do not prove equal performance. H3 anticipated a model-dependent balance between example overhead and correctness. P3_1-P0 yields three fewer passes on 0.5B but four more on 1.5B; this comparison combines examples and wording. P3_3 versus P0 yields five fewer passes on 0.5B and two more on 1.5B, with lower mean energy in both cases. Three examples are not universally better or worse than one. H4 anticipated extra cost with reasoning-style prompts and possible benefits on harder tasks. Extra cost is observed, but the expected hard-group advantage over P1 is not evident in the frozen calibration-failed group. H5 anticipated modest whole-request changes from longer code-only instructions. P5 and P1 have similar output lengths; P5's net increase is 2.91 mWh on 0.5B and 1.69 mWh on 1.5B, with the latter interval spanning zero. Neither equivalence nor input-phase cost is established.

These are explanation-requesting prompts applied to the same code models, not separate “reasoning” and “generational” architectures. Both generate token sequences; code execution occurs afterward in the test runner. Longer explanations require more token-generation steps, which can increase runtime and energy. The experiment does not inspect internal computation sufficiently to attribute errors to extra networks, overfitting, training preferences, or misunderstanding examples. A smaller model can generate a longer response; that means more decoding steps, not evidence of a different reasoning mechanism.

Model size is not a universal energy ranking. The 1.5B model has higher pass counts for every arm, but its mean net energy is lower for P1 and P5 and higher for several other arms. Per-token compute cost, response length and success jointly affect total cost. P1 is the strongest observed cost-per-correct candidate on both models; P3_3 is the cheapest per attempt on 0.5B; P3_1 has the highest observed accuracy on 1.5B. None establishes a universally optimal prompt or a guaranteed accuracy-energy trade-off for all workloads.

External validity is limited by one laptop, one model family, one quantization, 40 benchmark tasks, one main observation per task/arm/model, possible benchmark contamination, and a fixed output cap. Energy excludes testing, retries, whole-device consumption and training. Post-hoc contrasts are exploratory, intervals are pointwise, drift is systematic, and the failure annotations have no independent second-review agreement

estimate. Lower CPU core counts, cloud deployments, larger models and different domains were not tested. This evidence supports a bounded empirical case study rather than broad environmental or financial savings claims.

9.1. Comparing the three prompt categories

The categories organize the seven existing arms; they are not different model architectures or pooled treatment groups. Category A contains task instructions and generation (P0/P1/P5). Category B adds original examples before generation (P3_1/P3_3). Category C requests explanation or step-by-step reasoning before code (P2/P4). The final comparison sheet shows each arm separately on both models, including all supplementary views requested for this study.

Within Category A, P1 has the lowest observed output length and mean net energy on both models: 227.5 tokens and 26.61 mWh on 0.5B, and 71.1 tokens and 23.59 mWh on 1.5B. P0 uses more output and energy without a higher aggregate pass count. P5 adds 32 mean input tokens relative to P1, but its output is only 7.1% longer on 0.5B and 3.6% longer on 1.5B. P5 passes two fewer tasks on each model. These observations do not establish that longer instructions necessarily cause hallucination or poorer accuracy.

Within Category B, three examples produce fewer output tokens and lower mean net energy than one example on both models: 194.2 versus 357.8 tokens and 25.18 versus 41.05 mWh on 0.5B; 98.2 versus 130.2 tokens and 33.84 versus 40.94 mWh on 1.5B. In both cases, P3_3 passes two fewer tasks than P3_1 (19 versus 21; 28 versus 30). P3_3 therefore lowers per-attempt cost here while P3_1 has the higher pass count. Relative to P0, both example-guided arms have fewer successes on 0.5B and more on 1.5B. This model-dependent result contradicts a universal claim that examples always help or always waste resources.

Within Category C, P2 and P4 both produce long responses. On 0.5B, they pass 22/40 each; P4 uses slightly fewer tokens and lower energy (520.4 versus 531.0 tokens; 57.28 versus 59.13 mWh). On 1.5B, P4 passes one additional task (27 versus 26) but uses more output and energy (526.5 versus 500.1 tokens; 136.34 versus 128.29 mWh). Thus P4 is not uniformly worse than P2. Both have higher per-attempt and per-correct energy than P1 in this sample.

Comparing representatives P1/P3_3/P2 across categories illustrates why the metric matters. On 0.5B, mean net energy is 26.61/25.18/59.13 mWh and passes are 24/19/22; net energy per correct is 44.34/53.01/107.50 mWh. P3_3 is cheapest per attempt, while P1 has the lower cost per correct. On 1.5B, corresponding mean net energies are 23.59/33.84/128.29 mWh, passes 29/28/26, and net energy per correct 32.53/48.34/197.37 mWh. P1 is the strongest observed representative on these three metrics. These selected arms do not establish that every prompt in a category shares the same performance.

10. Future work and questions for prospective supervisors

The next useful study should prioritize measurement validity: repeated randomized blocks across days, comparable quiet baselines, timestamped thermal/background-load checks, and a small wall-meter validation. A factorial prompt experiment could hold the exact instruction fixed while varying demonstrations and output-format constraints separately. An independent held-out task set should assess whether any prompt-rewriting method preserves intent and generalizes, counting the rewriter's own cost.

A separate prefix-cache experiment could compare controlled shared prefixes with and without reuse, holding model residency and output workload constant and verifying actual cache behavior. This is distinct from conversation history or “forgetting.” CPU deployment work could then vary thread count, quantization and batch/concurrency settings under fixed quality targets. Praise, urgency and feedback experiments are lower priorities until measurement and prompt controls are stronger. No new experiment is run or registered by this proposal.

1. Is pairing by task sufficient here, and would you recommend a crossed task-by-session design or a hierarchical model for a follow-up?

2. How should we handle the observed idle-baseline drift: repeated local baselines, a time-varying model, gross-energy emphasis, or external power measurement?

3. Which next experiment would be most informative: a factorial examples/output-format study, controlled prefix caching, or CPU inference optimization at a fixed quality target?

11. Conclusion

The main lesson from these runs is that a longer answer did not necessarily produce better code. On the 1.5B model, the step-by-step prompt P4 averaged 526.5 output tokens and passed 27 of 40 tasks. The code-only prompt P1 averaged 71.1 tokens and passed 29. That is a clear difference in response length, but two extra passes are not enough to establish a reliable accuracy advantage.

Another useful finding came from the 0.5B model. P3_3 had the lowest mean energy per attempt, yet passed only 19 tasks. P1 passed 24 and had a lower energy cost per correct solution. Looking only at cheap responses would have missed that distinction.

We would take these observations forward by measuring useful results, not just output length or cost in isolation. This remains a small experiment on one machine, and the baseline drift makes the absolute net-energy values uncertain. A follow-up needs more stable power measurements, repeated sessions and prompts that vary one feature at a time before we can make broader recommendations.

AI assistance

AI tools helped set up the experiment and run the registered Ollama requests automatically. They also assisted with figures and the wording of this report. The failure annotations were AI-assisted and were checked against the saved responses; no independent second annotator was used.

References

[1] Roy Schwartz, Jesse Dodge, Noah A. Smith and Oren Etzioni. Green AI. arXiv:1907.10597, 2019 preprint. https://arxiv.org/abs/1907.10597

[2] Soham Poddar, Paramita Koley, Janardan Misra, Niloy Ganguly and Saptarshi Ghosh. Towards Sustainable NLP: Insights from Benchmarking Inference Energy in Large Language Models. NAACL, 2025, pp. 12688-12704. DOI: 10.18653/v1/2025.naacl-long.632. https://aclanthology.org/2025.naacl-long.632/

[3] Mark Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021. https://arxiv.org/abs/2107.03374

[4] Binyuan Hui et al. Qwen2.5-Coder Technical Report. arXiv:2409.12186, 2024. https://arxiv.org/abs/2409.12186

[5] Ollama. Generate API documentation. https://docs.ollama.com/api/generate

[6] HWiNFO. Hardware monitoring and sensor logging. https://www.hwinfo.com/

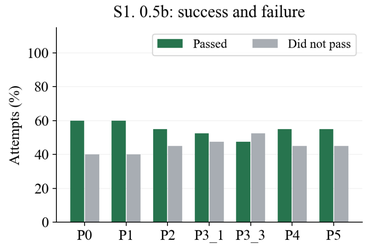
Appendix: retained evidence and software

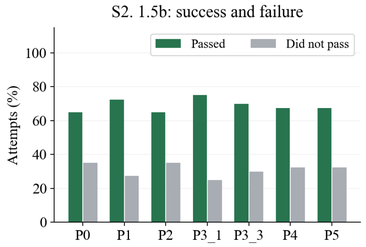
The primary analysis uses stage 1 of results/p4-full/full.csv; the CSV contains all 840 requests and explicitly marks stage. Raw responses, request metadata, tests and sensor logs remain in the project archive. The full CSV SHA-256 is b426f6bb5e79af91e437623e6692b9b2f41efc7aad7faec99ef50f81dba6e0e0. Generation source commit: 041f169. New post-hoc outputs are under results/p4-revision, including paired differences, ratio intervals, baseline scenarios, repeat diagnostics and source hashes. Public review exports are in artifacts/: all 840 rows, saved response bodies, metadata with local account paths redacted, CPU package power readings and supporting tables. Original local journals remain unchanged; export provenance records source and public hashes. Reduced sensor exports omit unrelated telemetry and are not byte-identical raw logs.

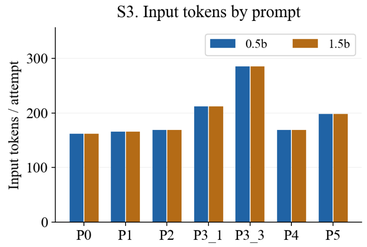
The HumanEval archive uses revision 6d43fb980f9fee3c892a914eda09951f772ad10d under MIT; exact model tags were checked under Apache 2.0. Local model SHA-256 digests: 0.5B, 4ff64a7f502a08b7616edb8ca0a79eb1853fc363d842b7df4b46915d11a3fb09; 1.5B, d7372fd828518a4d38b1eb196c673c31a85f2ed302b3d1e406c4c2d1b64a0668. Exact prompt assembly, original examples, frozen tasks and calibration labels are retained in docs/PROMPTS.md and experiments/config.py. Detailed environment and safeguards are recorded in docs/analysis/P4_SETUP.md and docs/SAFETY.md.

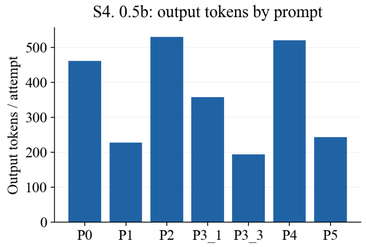
Study dependencies include NumPy 2.2.6, pandas 2.2.3, SciPy 1.15.3, Matplotlib 3.10.3 and psutil 7.0.0; tests use pytest 8.3.5 and Ruff 0.11.13. CodeCarbon 3.0.4 remains pinned for historical pilot reconstruction but is disabled in full-run scoring. PDF authoring uses a separate bundled runtime and does not change the study environment.

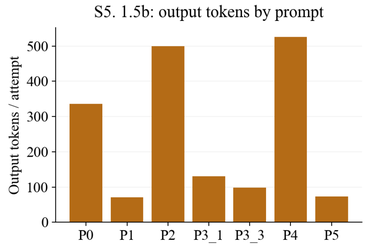
Supplementary comparisons: individual prompts within the three categories

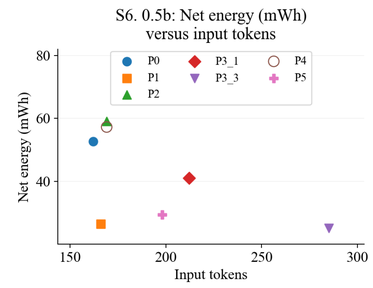


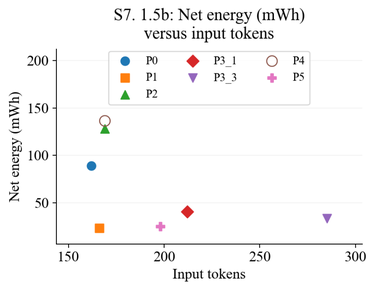


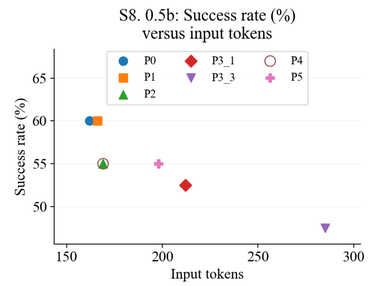


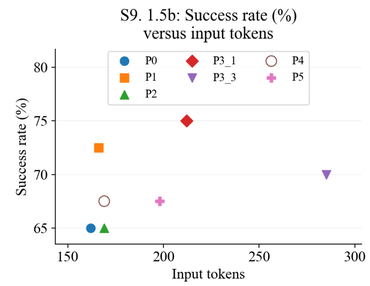


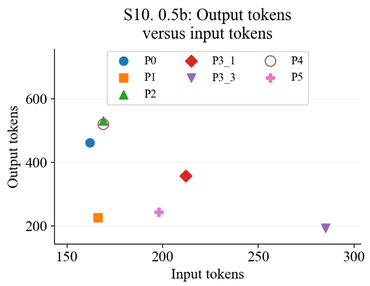


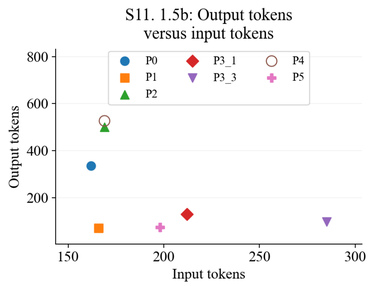


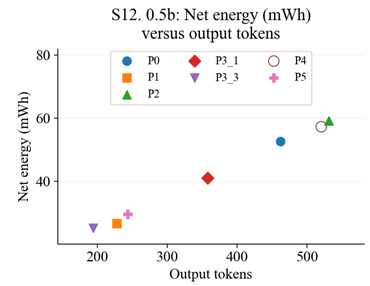


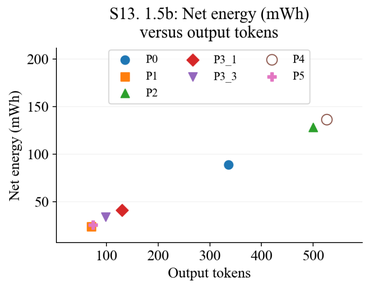


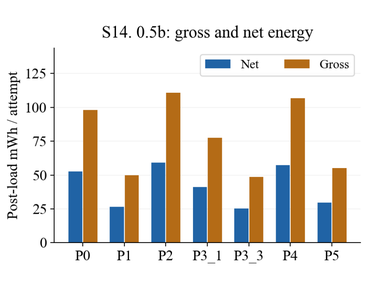


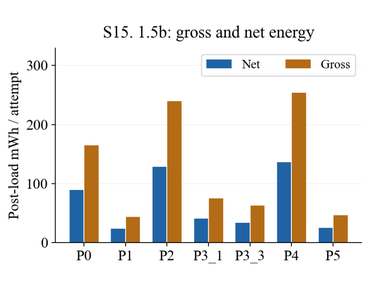


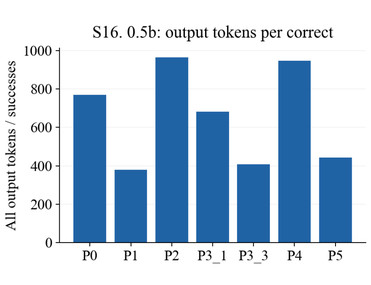


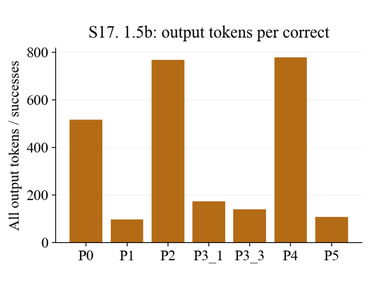


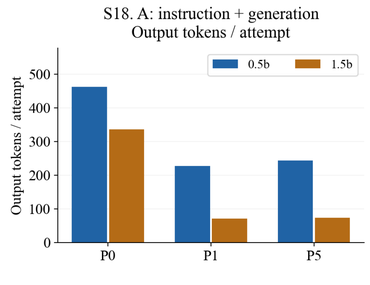


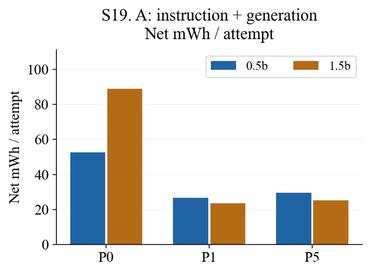


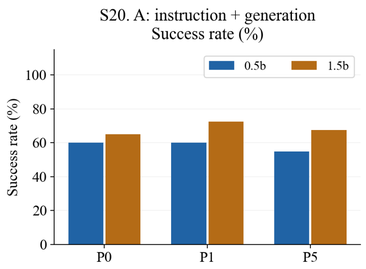


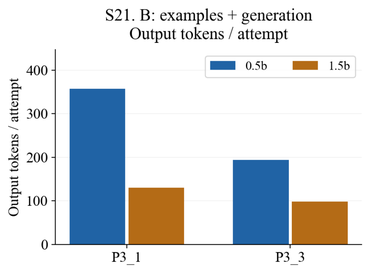


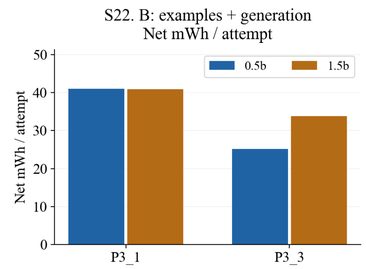


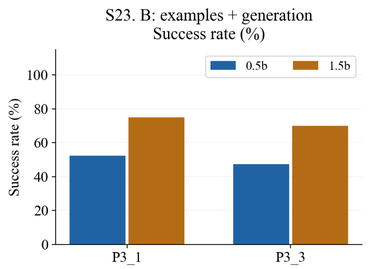


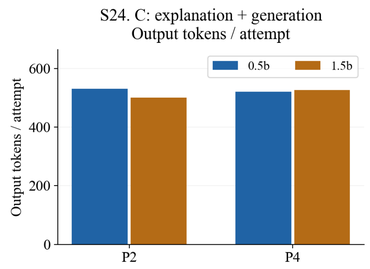


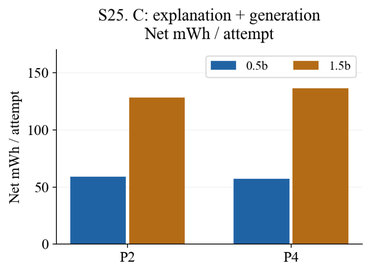


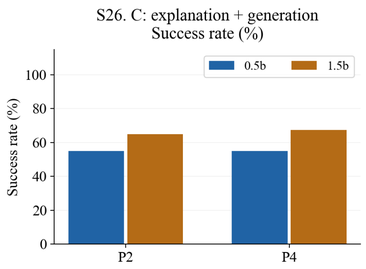


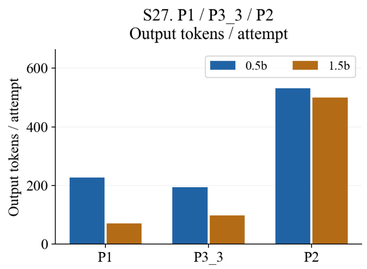


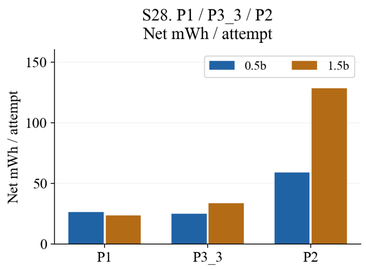


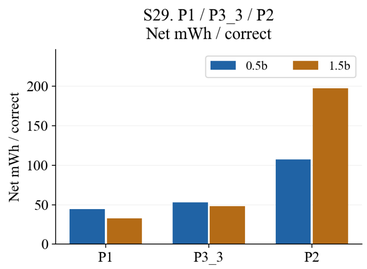


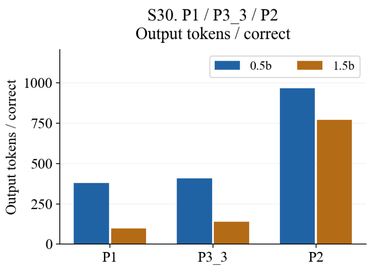












Stage 1: 40 tasks per model/arm. Point estimates; no category pooling. HWiNFO CPU Package Power, trapezoidal, approximate post-load.
Net uses block-start idle 17.553 W; gross is unsubtracted. All rows have baseline drift. Per-correct ratios include failed-attempt costs; main Figure 2 has energy-ratio intervals.